Electrical Engineering Technology (EET)

Embedded Plant Automation, Control, and Telemetry

By
Jack Watt

Senior Technical Report
Electronics Engineering Technology (EET)
2026

New Brunswick Community College – Saint John Campus
P.O Box 2270, Saint John, NB, Canada E2L 3V1
Phone: (506) 658 – 6600 Fax: (506) 643 – 2853
www.nbcc.ca

Departmental Disclaimer

This student's Applied Research Project – consisting of this Report and a Formal Presentation received a passing grade. **However, this report may contain errors, omissions, and inaccuracies.**

Engineering Technology students seeking examples of proper Report techniques should consult any guide materials from your Communication course instructor, as well as your course textbook.

Jack Watt
900 Grandview Avenue

Saint John, NB

E2J 4C5

February 20, 2026

Saadmaan Rahman
950 Grandview Ave

Saint John, NB
E2J 4C5

Dear Saadman Rahman,

As part of the requirements for the Electronics Engineering Technology program at NBCC, I am submitting to you my Senior Technical Report. The project, which you approved, on November 14, 2025, features an automation system to aid in the growth and monitoring of various plants. The system will allow the end user to input the optimal growing conditions for whichever plants they want to grow.

The objective for this project is to build a physical system that will monitor soil moisture, air humidity and temperature, and pump water based on the soil level. There is also a requirement to turn the lights on and off on a schedule, with the possibility of including a light level sensor to be able to compensate for natural light. This would allow for the system to function on natural light when available but have the backup LED grow lights when needed.

A core part of this project is internet connectivity, allowing for the modification of the running configuration and parameters without taking the system offline, as well as the ability to log sensor data over time on a web server and then make decisions based on that data (e.g. being able to modify how much water is given to the plants when the soil is low based on various factors). This enables automation beyond simple programming logic and supports the use of a sliding window to determine optimal operating parameters.

I wish to extend my gratitude to you as well as Meghan Hayes, who helped with determining the scope of the project and advised on how to write the technical report. I have encountered many problems to which your help has been instrumental in helping decide the correct forward path for the project. I would be glad to answer any questions about this project and can be reached through email at jack.a.watt@gmail.com.

Sincerely,

space for signature

Jack Watt

Jack Watt
900 Grandview Avenue

Saint John, NB

E2J 4C5

February 20, 2026

Meghan Hayes
950 Grandview Ave

Saint John, NB
E2J 4C5

Dear Meghan Hayes,

As part of the requirements for the Electronics Engineering Technology program at NBCC, I am submitting to you my Senior Technical Report. The project, which was approved by Saadman Rahman, on November 14, 2025, features an automation system to aid in the growth and monitoring of various plants. The system will allow the end user to input the optimal growing conditions for whichever plants they want to grow.

The objective for this project is to build a physical system that will monitor soil moisture, air humidity and temperature, and pump water based on the soil level. There is also a requirement to turn the lights on and off on a schedule, with the possibility of including a light level sensor to be able to compensate for natural light. This would allow for the system to function on natural light when available but have the backup LED grow lights when needed.

A core part of this project is internet connectivity, allowing for the modification of the running configuration and parameters without taking the system offline, as well as the ability to log sensor data over time on a web server and then make decisions based on that data (e.g. being able to modify how much water is given to the plants when the soil is low based on various factors). This enables automation beyond simple programming logic and supports the use of a sliding window to determine optimal operating parameters.

I wish to extend my gratitude to you as well as Saadman Rahman, who helped with determining the scope of the project and advised on how to write the technical report. I have encountered many problems to which your help has been instrumental in helping decide the correct forward path for the project. I would be glad to answer any questions about this project and can be reached through email at jack.a.watt@gmail.com.

Sincerely,

space for signature

Jack Watt

Embedded Plant Automation, Control, and Telemetry

Senior Technical Report (1261-3286)

Embedded Plant Automation, Control, and Telemetry

By
Jack Watt

Applied Research Project Report
Electronics Engineering Technology (EET)
2026

New Brunswick Community College – Saint John Campus
P.O Box 2270, Saint John, NB, Canada E2L 3V1
Phone: (506) 658 – 6600 Fax: (506) 643 – 2853
www.nbcc.ca

Departmental Disclaimer

This student's Applied Research Project – consisting of this Report and a Formal Presentation received a passing grade. **However, this report may contain errors, omissions, and inaccuracies.**

Engineering Technology students seeking examples of proper Report techniques should consult any guide materials from your Communication course instructor, as well as your course textbook.

TABLE OF CONTENTS

LIST OF FIGURES iv
LIST OF TABLES v
LIST OF APPENDICES vii
EXECUTIVE SUMMARY ix
Statement of problem x
1. Introduction 1
2.0 Project Overview 2
2.1 Objectives 2
2.2 Project details 2
2.3 Physical Build 4
2.4 Electronics 5
2.5 Software 5
wscat 6
Tee-Object 6
2.6 Testing 6
2.7 Components with cost breakdown 7
2.8 System Design and implementation 7
3.0 Research 8
Works Cited 9
Appendix C: Code 9
APPENDIX D: User manual 27
1: The physical system: 28
2: Configuring the code: 29
3: The Software and user interface 30
3.1 -Wscat 30
3.2 – Tee-Object 30
APPENDIX E: Wiring Diagram and Block Diagram 31
Wiring Diagram 32
Block Diagram 33

LIST OF FIGURES

LIST OF TABLES

LIST OF APPENDICES

APPENDIX A: ARP Approval Form
APPENDIX B: Progress Report
APPENDIX C: Code
APPENDIX D: User manual
APPENDIX E: Wiring and Block Diagram

EXECUTIVE SUMMARY

This project presents the design and implementation of an embedded system for automating the growing of plants. The objective was to develop a functional prototype capable of monitoring environmental conditions and automating watering and lighting. The system was designed to integrate hardware control, embedded programming, and computer networking within a single platform.

The solution is built around an ESP32 microcontroller platform with integrated Wi-Fi capability. Monitoring is achieved using soil moisture sensors, air humidity and temperature sensors, an ambient lighting sensor, and a remote web server that will be making the decisions based on data communications from the ESP32.

A key feature of the system is remote connectivity. The ESP32 is in constant communication with a web server to retrieve operating parameters and configuration settings. This architecture separates control logic, data storage, and data analysis, allowing for remotely modifying the system parameters based on optimization strategies that have been implemented.

The prototype successfully integrates sensing, scheduling and wireless communication into a single embedded system. All primary functions – automated watering, scheduled lighting, and Wi-Fi communications -- are operational. The project demonstrates practical application of industrial control principles, power regulation design, embedded systems programming, and network communication within a scalable automation framework.

STATEMENT OF PROBLEM

Plant care is a science, which makes manual care inconsistent and time-consuming. Growers who choose manual care might forget watering schedules, over or under water plants, or fail to provide consistent lighting. Existing commercial automation systems can be expensive and aren't viable for small-scale applications. Therefore, a low-cost customizable embedded automation system is needed.

This system can be built for under $100 if only considering the electronics and not considering the enclosure, plants, or soil. This system also includes the source code as well as a wiring guide, which will allow the end user to customize for their needs.

1. INTRODUCTION

Manual plant monitoring is both inefficient and inconsistent. With automation there can be improved reliability and scalability. This project explores implementing an embedded control system as a solution to monitoring and automation.

The system integrates sensors to measure environmental variables such as soil moisture, temperature, and humidity, allowing for continuous real time data collection and decision making. Based on the inputs provided, the system can make autonomous control decisions, such as turning on and off the lights and activating the water pump.

The system also integrates networking to allow for real time data retrieval and viewing, as well as the ability to change the running conditions of the system. It can be quite inconvenient to have to disconnect a system to update the conditions of the running program, which is why part of this system involves an interface to edit the conditions remotely without having to take the system offline.

2.0 PROJECT OVERVIEW

2.1 Objectives

The objectives of the project are as follows:

- Build a test circuit and write code to use a temperature sensor, humidity and moisture sensor, water level sensor, and ph sensor individually.
- Build a test circuit using a relay to run a water pump and lights
- Combine the circuits to pump water based on low moisture in soil and turn the lights on and off on a schedule
- Display the sensor values on an LCD
- Write a web server or use thingspeak to be able to monitor the sensors remotely
- Write code to be able to modify the program conditions remotely
- Build complete project enclosure to house all electronic components
- Graph sensor data over time to verify configuration is optimal.

2.2 Project details

- Enclosure size

The size of the enclosure is 80 gallons. At this size it would be optimal to have another soil moisture sensor, which would split the enclosure into three zones. If the project is being rebuilt or transferred to another enclosure, it would ideally be smaller or would include another soil moisture sensor and a second water pump.

- Watering reservoir

The water reservoir can be of any size, the one included in the project is a 4L water bottle, but the larger the reservoir, the longer the system can be left unattended. For an industrial application of this project is would be ideal to replace the water pump with a direct connection to water, although a nice thing about the 4L container is that even if it can't overflow the enclosure.

- Sensor placement

The sensors are on each side of the enclosure, but they can be moved around as needed. Ideally there would be a third in the project, but as of now only two are present.

- Wiring layout

The wiring layout separates the low-voltage control from the higher-power loads. The ESP32 handles sensing, control logic, and communication. Solid state relays are used to isolate and switch the water pump and lighting circuits.

- Plant species tested

There was a very limited number of plants tested. This system isn't providing off-the-shelf configurations for growing any specific plant. The system provides an interface that allows the end user to set how moist the soil should be as well as how much lighting it should receive. This means that the system would need to be upgraded to allow for growing plants that require controlling other factors such as air humidity, temperature, C02 levels, feeding of nutrients, etc

- Power sources used

The power sources used in this project are a 5v wall outlet USB charger, and a 12v and 6v rechargeable battery. Ideally the system would only use one power source. There are benefits to both battery and using the wall outlet. Hypothetically a system could run for days on a single battery, which would allow for a control system in a greenhouse that doesn't have an available wall outlet. Using the wall outlet would allow for a system that runs for much longer without human intervention.

- Communication method

Communication with the system is achieved through the built-in Wi-Fi capability of the ESP32. A WebSocket connection allows remote commands to be sent to the system and operating data to be viewed in real time from a computer terminal.

2.3 Physical Build

The physical prototype was constructed using a clear plastic storage tote as the plant enclosure. This material was selected because it’s inexpensive and available at most hardware stores. It being transparent was chosen to be able to visually inspect the plants from the sides rather than just top down. For plants that require a lot of lighting, it would be best to use an enclosure with reflective siding to increase the amount of lighting the plants get.

The electronic control components were mounted outside of the primary growing area to reduce exposure to moisture and humidity. This includes the ESP32, relays, and most of the wiring connections. The only sensors in the enclosure are the soil sensors. They will most likely have to be replace at some point because they are only water resistant up to a point on them. There is exposed electronics on the soil moisture sensors that aren’t meant to be wet or humid, using some kind of heat shrink to seal them would be ideal, but that wasn’t done here.

Lighting was mounted above the plant area to simulate grow-light conditions. The light position can be adjusted depending on plant height and coverage requirements.

During assembly, effort was made to shorten wire lengths and organize connections into a compact layout. Further improvements would include cleaner cable routing and a PCB for power distribution.

2.4 Electronics

The electronics portion of this project is centered around the ESP32 microcontroller, which was selected because it provides built in Wi-Fi capability, multiple GPIO pins, analog inputs, and more than enough processing power for the control logic required by the system. The ESP32 acts as a controller for reading sensors, making decisions, driving outputs, updating the display, and handling wireless communications.

Two soil moisture sensors are connected to analog input pins of the ESP32. These sensors provide raw digital values which are converted in software into approximate moisture percentages. A humidity and temperature sensor is connected through the I2C communication bus and provides environmental data from inside the enclosure.

Outputs from the ESP32 are used to control two solid-state relays. One relay switches the water pump circuit and the second relay switches the lighting circuit. Solid-state relays were selected because they provide electrical isolation between the low-voltage control side and the higher-current loaded side. Isolation means that the 12v or 6v power sources won't be able to interfere with the control side. The ESP32 might be able to handle the 6v power, but it can't handle the 12v power. It also can't handle the current loads from the light or the pump.

A 1.8 inch TFT display is connected through the SPI interface and is used to show live system information such as time, moisture levels, humidity, temperature, lighting status, and the pump mode.

2.5 Software

Overview:

The software for this project was written in C++ using the Arduino IDE for the ESP32 platform. The program is responsible for reading sensor data, controlling outputs, updating the display, managing schedules, and handling wireless communication.

The main program loop reads the soil moisture sensors, humidity sensors, and temperature sensor. Raw readings are converted into percentages using calibration points determined during testing. These values are used by the control logic to determine whether watering is required.

Automatic watering logic is included in the software. If the soil moisture level falls below a user-provided threshold, the system activates the pump for a timed interval and then waits five minutes before pumping again. This was a simple way to ensure the plants aren't overwatered.

Lighting control is also handled by the software. The user can enter a start time and duration through the remote interface. The ESP32 synchronizes its time through network time servers and automatically turns the lights on and off according to the programmed schedule.

Wireless communication is implemented using a WebSocket server running on the ESP32. This allows remote terminal connections to send commands such as turning lights on and off, manually controlling the pump, enabling automatic watering, and changing schedules without uploading new code.

The TFT display is updated in real time to provide local status information. Sensors readings, output states, time, an operating modes are shown directly on the screen so the system can be monitored without using the remote terminal.

wscat

Wscat is a version of netcat that works directly with web sockets. This piece of software was chosen because it's easy to use and because the websocket will hold open a connection rather than have the system try to make a new connection every time there is data to send or receive.

Tee-Object

Tee-Object is a command line problem that allows the output of a program to be displayed on screen as well as logged to a file. This allows the system to show the user the sensor data in the terminal, as well as log it in a file so it can be graphed or reviewed later.

2.6 Testing

- Moisture control

With the moisture soil sensors, testing was done to find the sensor data that corresponds with the sensor being dry and wet. It was found that the digital value for the sensors read close to 1500 when submerged in a water and close to 3000 when exposed to dry air.

The system is set to water the plants at 50% relative moisture, which is 2250 digital sensor value. It has been tested and determined that the water pump will activate at the 50% point, but testing wasn't performed to see if watering at the 50% point is optimal for any specific plant. It's just been verified that if the user puts in a point at which the pump should activate that it will work.

- Lighting Schedule

The lighting schedule was tested multiple times. If the command "start 1:20 180" is run in the terminal, the system will turn the lights on at 1:20 for 180 minutes. Edge cases were also tested, such as "start 22:00 360" which turns the lights on starting at 10 pm for 6 hours. The reason for this test was to make sure a lighting schedule works if it spans multiple days.

- Wi-Fi commands

To test the wifi commands, commands were sent from the terminal to the system and then verified that they worked as intended. Incorrect commands were sent to test that the system won't crash if the supplied command isn't available.

- Display output

The display output was tested to ensure the values were aligned correctly, showed correct sensor data, and were easy to read.

2.7 Components with cost breakdown

- DFRobot SHT31 Weather-Proof Temperature & Humidity Sensor - $32[4]
- DFRobot Gravity Moisture Sensor - $9 -[5]
- 2x DC to DC Solid State Relay - $25 -[6]
- DC Water Pump -$13 -[7]
- Grow Light 15W Sunlike Full Spectrum LED Plant Grow Light -$15 - [8]
- ESP32 - $15 - [9]
- ESP32 screw terminal - $13 - [10]
- 1.8 inch RGB TFT display - $12 - [11]
- Mastercraft Heavy Duty Stackable Storage Box with Lid – $20 -[12]

Total cost: $154

2.8 System Design and implementation

The system was designed as an integrated embedded control platform capable of monitoring plant conditions and automatically responding to changes in the plants conditions. The overall design combines sensing hardware, output control devices, embedded software, and wireless communication into a single system centered around the ESP32 microcontroller.

The final implementation demonstrates how sensing, automation, scheduling, and networking can be combined into a practical low-cost embedded control system. The design is modular and can be expanded in future versions with additional sensors, improved power management, and cloud-based data logging.

3.0 RESEARCH

Most of the research I did for this system consisted of how to connect and control the project remotely. I wanted to be able to set the running conditions of the project remotely, to be able to update the lighting and watering cycle without having to take the system offline to push new code to the ESP32.

Most of my components were cheap and not much thought went into picking them. I wanted to measure the soil moisture level, so I bought sensors for that. There wasn't any effort put into comparing different sensors, components were selected based on availability through robotshop had because I have bought from them before.

I bought solid state relays. I already had some relays, but my advisor recommended solid state relays on the grounds that they are more reliable. I ended up picking two solid state relays from amazon that were cheap.

I picked the ESP32 because I already had it and because it has built in Wi-Fi.

- Capacitive vs Resistive soil sensors

Capacitive sensors are more accurate and last longer, but they are more expensive and susceptible to salt and other nutrients. Having a better grasp on the chemistry involved with growing plants would be ideal when using capacitive soil sensors.

Resistive sensors are the better option for a prototype like this one. They are cheap and don't require calibration.

- Lighting

The correct lighting depends on the plants being grown. For general plant care, the cheap lighting I bought is fine, but it would be advised to switch the lighting for something else if growing a specific plant. Plants typically respond to a specific frequency of light. I went with the lighting I did because it was inexpensive and easy to work with. An upgraded version of this system would include lighting for the specific use case the end user has.

- Communication

I believe the ideal communication method would have been MQTT, but I went with Wscat because my background includes using terminal applications such as wscat. I used it because it was easiest for me. With more time or for an upgrade, I would recommend using a managed service provider to host MQTT.

WORKS CITED

[1] "SHT31 Weather-proof Temperature & Humidity Sensor Wiki - DFRobot," *Dfrobot.com*, 2019. https://wiki.dfrobot.com/SHT31_Temperature_Humidity_Sensor_Weatherproof_SKU_SEN0385

[2] Sapry EL Gammal, "ph," *Scribd*, 2026. https://www.scribd.com/document/609479894/ph (accessed Feb. 20, 2026).

[3] "RB-Dfr-191 Immersible Water Pump and Water Tube," *https://cdn.robotshop.com/media/d/dfr/rb-dfr-191/pdf/datasheet-fit0200.pdf*.

[4] "DFRobot SHT31 Weather-Proof Temperature & Humidity Sensor," https://ca.robotshop.com/products/dfrobot-sht31-weather-proof-temperature-humidity-sensor

[5] "Gravity Moisture Sensor" https://ca.robotshop.com/products/gravity-moisture-sensor-corrosion-resistant

[6] "High Speed, Anti Vibration DC DC Solid State Relay SSR60DD 60A 332VDC to 5200VDC SSR" https://www.amazon.ca/dp/B0DKXYT9KF?ref=ppx_yo2ov_dt_b_fed_asin_title

[7] "Water pump" https://www.robotshop.com/products/water-pump

[8] "Grow Light 15W sunlike full spectrum LED plant grow light" https://www.amazon.ca/Spectrum-Waterproof-Dimmable-Adjustable-Gooseneck/dp/B08T62JX17/140-8483008-6297306

[9] "ESP32-WROOM-32" https://www.amazon.ca/ESP-WROOM-32-NodeMCU-Bluetooth-Development-Microcontroller/dp/B0CHBMFJBQ?crid=3P86SYFD78BFK

[10] "ESP32 Screw Terminal board" https://www.amazon.ca/Header-Terminal-Narrow-Program-Support/dp/B0D1TMC4LW

[11] "1.8 inch RGB Character TFT Display Module 8Pins " https://www.taydaelectronics.com/1-8-inch-rgb-character-display-module-8pins.html

[12] "Mastercraft Heavy Duty Stackable Storage Box with Lid" https://www.canadiantire.ca/en/pdp/mastercraft-heavy-duty-stackable-storage-box-with-lid-102-l-black-blue-1426045p.html

APPENDIX A: ARP Approval Form

Source: Original

APPENDIX B: Progress Report

TO: Saadmaan Rahman
COPIES: Meghan Hayes
FROM: Jack Watt
DATE: 2026/02/06
SUBJECT: **NBCC-SJ EET2 Senior Project – Progress Report #2**
Project Title: Embedded Plant Automation, Control, and Telemetry
Period Covered: 2026/01/04 – 2026/02/06

PROJECT OBJECTIVE/OVERVIEW

As part of my plant growing and monitoring project, I am responsible for designing and implementing both the software and physical components of the system. I chose to work on this project because it incorporates many of the skills I have learned in my EET courses, as well as my previous experience. This project involves circuits and programming, which are core topics in my program. It also makes use of sensors, pumps, and lighting systems that align closely with the industrial control course I was taking with Rejean Bariault. My previous experience is in networking, computer programming, and web automation.

The objective for this project is to build a physical system that will monitor soil moisture, air humidity and temperature, and pump water based on the soil level. There is also a requirement to turn the lights on and off on a schedule, with the possibility of including a light level sensor to be able to compensate for natural light. This would allow for the system to function on natural light when available but have the backup LED grow lights when needed.

A core part of this project is internet connectivity, allowing for the modification of the running configuration and parameters without taking the system offline, as well as the ability to log sensor data over time on a web server and then make decisions based on that data (e.g. being able to modify how much water is given to the plants when the soil is low based on various factors). This enables automation beyond simple programming logic and supports the use of a sliding window to determine optimal operating parameters.

I have chosen an ESP32 as the platform to control the system, as it has built-in Wi-Fi.

I have yet to fully decide on what the enclosure will be or how big it will be, but I'm leaning

towards a 40-gallon clear storage tote or larger. Ideally, I would use something like a terrarium for reptiles, as I think that would be the most ideal enclosure for my project, but for monetary reasons and practicality of this proof of concept, I think it would be best to build a system in something more affordable.

The benefit of the clear storage tote is that I can cut into the side of it and mount my in-soil sensors, which allows them to be in contact with the soil without exposing the electrical connections to the humid environment inside the enclosure.

WORK COMPLETED

So far, I have reached the first milestone, which includes having all my sensors, my pump, and my lighting running at the same time. When the soil moisture is low, it pumps water into the plants and turns on and off the lights on an 8-hour on, and 16-hour off cycle.

I've started to make my wiring neater and have cut my wires down to optimal lengths to be able to have the entire project contained in a small box. Eventually some of the wiring will need to be replaced because the sensors will be placed inside an enclosure with plants, while the rest of the system will be mounted to the enclosure in some way.

I currently have the Wi-Fi on the system working, which periodically scans a static web page looking for the running configuration of the system. Currently, the system is just checking for a single bit which indicates whether it should pump water. This simple code is the starting point for the rest of the code which will read the entire page, parsing it into lines, and then overwriting the default conditions of the system.

Background Material Collected:

a) Read the data sheets and example code for my SHT31 humidity/temperature sensor

b) Read an example code for the ESP32 Wi-Fi and HTTP libraries.

c) Consulted with instructor Saadmaan about the use of a power circuit, which will use a 5v voltage regulator with a 6- and 12-volt battery to get both 5 volts and 3.3 volts. I'll need at least 3.3v, 5v, and 9v+. Saadmaan helped to make the decisions on how to achieve this.

d) Consulted prior instructor on industrial control principles, and ideas on how to achieve the project objectives.

PROBLEMS ENCOUNTERED

Most of the problems I've encountered had to do with the scope of my project. I pitched my idea without fully understanding the cost of what I wanted to build. Before the proposal was accepted, Saadmaan allowed me to scale the project back and remove the heating and fans I originally wanted. The reason these were removed is that in the humid environment I wanted to create, I would have needed special fans that can work while being wet, as well as I would have needed an enclosure that could handle heat. There is also the consideration that overheating the enclosure could be dangerous, even if I did have an appropriate enclosure. As a result, these aspects were removed to reduce cost and avoid potential safety risks.

The next problem I encountered is that the pH probe I bought was meant to be used with a solution made from distilled water and then maintained with a chemical that I wasn't sure if I had to buy or make myself. This was an oversight on my part, because I meant to buy a pH sensor that could be stuck directly into the soil and give periodic readings. My advisor Saadmaan allowed me to remove it from my project.

The final problem I encountered is that while getting everything connected, I shorted my 12-volt battery and destroyed one of my solid-state relays.

Future Work

There is still a lot that could be done here.

Remote rather than local

Currently, there is a local connection over Wi-Fi to manage the automation system. Having the webserver run in the cloud rather than on a local machine would be a big improvement to the system. The local network works fine for demonstration, but moving the system to a web server hosted by amazon web services would be ideal. Right now the system can only be modified while on the same network as the automation system.

Watering system

An improved watering system would use a sprinkler to water the entire enclosure evenly. Or, another option would be a system that will allow water specific sections. Right now the system has a hose with holes in it to be able to water more than one area, but it doesn't look good and it waters the areas unevenly.

Case layout

The case currently is quite messy. I'm using terminal strips to have a common 5v rail and a common ground. It would be neater and easier to maintain if a PCB were used. It would also be useful to write wrap each individual sensors wires. Right now, the soil sensors have 3 wires twisted together. They run from electronics case to the plant enclosure.

Improved soil sensor durability

The soil sensors have exposed electronics that aren't corrosion resistant. They could be be protected with heat shrink tubing or there are more expensive soil moisture sensors that don't have exposed electronics.

Improved lighting

There is currently one light which can have its position adjusted, but it would need to be adjusted multiple times per day to get the optimal lighting to the entire enclosure. The solution could be to buy more lights or to mount the light on a servo motor to have it slowly change its position over the length of its lighting cycle.

The same light I used in the project comes in a version that has multiple adjustable lights on it. I would suggest starting with that, as it's probably the easier improvement to implement.

Eliminate multiple power sources

Right now there is a 12v battery, 6v battery, and USB charger. With a voltage regulation circuit, the water pump, lighting, and esp32 could be powered using the same source. It doesn't make sense to have a product that requires three power sources.

Fix the water level circuit

The water level sensor currently is not reliable. Sometimes it detects the water and sometimes it doesn't. The final version of my code has forced the system to always think the water sensor detects water, because I wanted my final demonstration to go well. I suspect the culprit is the lighting on the lab mixed with the fact it's quite a cheap sensor. A better-quality water sensor could be used. This is an important update because if the water pump runs without water in the reservoir would most likely burn out the water pump.

Critical Alerts

Being able to receive an email or text message when something in the system is outside of an expected value should result in the end user being emailed or text messaged. The water reservoir being empty could be one, as well as if the soil moisture sensors register the soil outside of ideal ranges.

Nutrients

Currently there is nothing in place to feed the plants automatically. It would be possible to put something in the water reservoir, but ideally there would be another pump that could pump in nutrients.

Separate logging from remote access

Currently there is a connection made from a command line terminal which allows for sending commands to the system as well as for receiving updates on the system. Ideally these would be separate to allow for data logging when there is no remote connection.

Measure ambient lighting

The system currently can be set to turn the lights on for a duration every day. With a sensor to measure ambient lighting, it would be possible to not run the grow light if the system is receiving adequate natural lighting. This would allow for the system to only be operational when there isn't enough natural lighting, which could save money.

Self-correcting behavior

The system currently will turn the water pump on for 5 seconds and then will wait 5 minutes to see if the moisture sensors have picked up the water added to the enclosure. This is a way to prevent overwatering, but with improved analysis, the system could use a PID loop to stabilize its outputs.

The main physical work I have left to do is to build a permanent enclosure for my plants and my system. All I need is the enclosure, as I have everything else. I'll just have to replace some wires to make things longer to reach from the system to the enclosure.

After I complete the physical work, everything left will involve creating a web server that my ESP32 can talk to. This will allow for data logging, as well as getting the parameters of the system. Any decisions on how to modify the parameters of the program will happen on the web server, which will separate the system running the project from any data analysis, logging, or storage. One of the benefits of this is that if my system loses power for any reason, none of the data will be lost.

CONCLUSION

The scope of the project slightly changed from the first proposal until the final demonstration. Partly because of issues that were encountered, but also to avoid potential issues that could occur. This was a project that tested and leveraged several areas of my study at NBCC. I chose to do a project on plant automation, because I saw it as an opportunity to get to program, build electronics, and try to incorporate the industrial control technology courses we had started around the beginning of this project. In the end I wouldn't consider the project to leverage as much of the ICT courses as I would have liked, but I am satisfied that I successfully measured multiple environmental factors and responded to some of them.

The problems encountered with the circuits and building took away some of my time from programming. I didn't initially intend to use AI to generate my websocket code, but as time went on in the program I saw it as a way to reclaim some of my lost time and focus more on the physical build and making the layout of the project as nice as possible. I still would like to improve the layout of the components in the enclosure, but how I have it now is about as good as it's going to get without rewiring parts of it, which I decided against. It seemed like a better plan to demonstrate the project with some messy wires than to end up with a non-functional project because I started rewiring at the last moment.

If I could go back and start over again I would put some time aside very early on to consider the layout of the final project. Being able to get that out of the way would have meant avoiding the mess I ended up with in the end, as well as I most likely wouldn't have short circuited my SSR.

There were some objectives that had to be changed over time. When I started out there was going to be heating and a fogger. I quickly talked with my advisors to see if I could change the scope of the project because I was concerned about running a heater as it could melt my plant enclosure and start a fire if I made a mistake. I also was concerned about using the fogger because not everything in my project is made to used in wet humid environments. I ended up dropping both of these requirements to ensure that my project wouldn't end up being lit on fire or damaged from water exposure.

I learned a lot from this project. There were times where I didn't know how to continue or what was wrong with my circuit. There were multiple times where my code for one sensor or output device would block the rest of the program from executing. Being able to tell when this was happening helped me progress.

APPENDIX C: CODE

This code is partly generated by AI. Almost everything for the communication between the esp32 and a terminal program was written by AI. Everything else was written by hand. All logic for the system was designed and implemented without AI.

```
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7735.h>
#include <Wire.h>
#include <DFRobot_SHT3x.h>
#include <WiFi.h>
#include <WebSocketsServer.h>
#include <time.h>

#define I2C_SDA 21
#define I2C_SCL 22

TwoWire I2CBus = TwoWire(0);
DFRobot_SHT3x sht3x(&I2CBus, 0x44);

#define TFT_CS   5
#define TFT_DC   2
#define TFT_RST  4

int lights_on = 1;
int water_on = 0;

int scheduledStartMinutes = 0;
int scheduledDurationMinutes = 0;
bool scheduleEnabled = false;

const int moistureTriggerPercent = 50;
const unsigned long wateringDurationMs = 5000;
```

```
const unsigned long wateringCooldownMs = 300000; // 5 minutes

bool autoWaterEnabled = true;
bool wateringPulseActive = false;
unsigned long wateringPulseStart = 0;
unsigned long lastWateringTime = 0;

Adafruit_ST7735 tft = Adafruit_ST7735(TFT_CS, TFT_DC, TFT_RST);

// WiFi
const char* ssid = "YOUR_WIFI";
const char* password = "YOUR_PASSWORD";

// WebSocket server on port 81
WebSocketsServer webSocket = WebSocketsServer(81);

int moisturePercent(int rawValue) {
  const int wetValue = 1500;
  const int dryValue = 3000;

  if (rawValue <= wetValue) return 100;
  if (rawValue >= dryValue) return 0;

  return (dryValue - rawValue) * 100 / (dryValue - wetValue);
}

void webSocketEvent(uint8_t num, WStype_t type, uint8_t * payload, size_t length) {
  switch (type) {
    case WStype_DISCONNECTED:
```

```
  Serial.printf("[%u] Disconnected\n", num);
  break;

case WStype_CONNECTED: {
  IPAddress ip = webSocket.remoteIP(num);
  Serial.printf("[%u] Connected from %d.%d.%d.%d\n", num, ip[0], ip[1], ip[2], ip[3]);
  webSocket.sendTXT(num, "ESP32 connected");
  break;
}

case WStype_TEXT: {
  String msg = String((char*)payload);
  msg.trim();
  Serial.printf("[%u] Received: %s\n", num, msg.c_str());

  int water_level = digitalRead(34);
  int soil_1 = analogRead(35);
  int soil_2 = analogRead(32);
  int soil1Percent = moisturePercent(soil_1);
  int soil2Percent = moisturePercent(soil_2);

  if (msg == "status") {
    String reply = "";

    struct tm timeinfo;
    if (getLocalTime(&timeinfo)) {
      reply += "time=";
      if (timeinfo.tm_hour < 10) reply += "0";
      reply += String(timeinfo.tm_hour);
```

```
  reply += ":";
  if (timeinfo.tm_min < 10) reply += "0";
  reply += String(timeinfo.tm_min);
} else {
  reply += "time=unavailable";
}

reply += ", lights_on=" + String(lights_on);
reply += ", water=" + String(water_level);
reply += ", pump_on=" + String(water_on);
reply += ", soil1_pct=" + String(soil1Percent);
reply += ", soil2_pct=" + String(soil2Percent);
reply += ", soil1_raw=" + String(soil_1);
reply += ", soil2_raw=" + String(soil_2);
reply += ", tempC=" + String(sht3x.getTemperatureC());
reply += ", humidity=" + String(sht3x.getHumidityRH());
reply += ", auto_water=" + String(autoWaterEnabled ? "on" : "off");

if (scheduleEnabled) {
  reply += ", schedule=";
  int schedHour = scheduledStartMinutes / 60;
  int schedMin = scheduledStartMinutes % 60;
  if (schedHour < 10) reply += "0";
  reply += String(schedHour);
  reply += ":";
  if (schedMin < 10) reply += "0";
  reply += String(schedMin);
  reply += " for " + String(scheduledDurationMinutes) + " min";
} else {
```

```
    reply += ", schedule=disabled";
  }

  webSocket.sendTXT(num, reply);
}
else if (msg == "lights on") {
  scheduleEnabled = false;
  lights_on = 1;
  webSocket.sendTXT(num, "OK lights on");
}
else if (msg == "lights off") {
  scheduleEnabled = false;
  lights_on = 0;
  webSocket.sendTXT(num, "OK lights off");
}
else if (msg == "water off") {
  water_on = 0;
  wateringPulseActive = false;
  autoWaterEnabled = false;
  webSocket.sendTXT(num, "OK water off");
}
else if (msg == "water on") {
  water_on = 1;
  wateringPulseActive = false;
  autoWaterEnabled = false;
  webSocket.sendTXT(num, "OK water on");
}
else if (msg == "auto water on") {
  autoWaterEnabled = true;
```

```
  webSocket.sendTXT(num, "OK auto water on");
}
else if (msg == "auto water off") {
  autoWaterEnabled = false;
  wateringPulseActive = false;
  water_on = 0;
  webSocket.sendTXT(num, "OK auto water off");
}
else if (msg == "schedule off") {
  scheduleEnabled = false;
  webSocket.sendTXT(num, "OK schedule off");
}
else if (msg.startsWith("start ")) {
  int firstSpace = msg.indexOf(' ');
  int secondSpace = msg.indexOf(' ', firstSpace + 1);

  if (secondSpace == -1) {
    webSocket.sendTXT(num, "invalid format, use: start HH:MM durationMinutes");
    return;
  }

  String timeStr = msg.substring(firstSpace + 1, secondSpace);
  String durationStr = msg.substring(secondSpace + 1);
  timeStr.trim();
  durationStr.trim();

  int colonIndex = timeStr.indexOf(':');
  if (colonIndex == -1) {
    webSocket.sendTXT(num, "invalid time, use HH:MM");
```

```
  return;
}

String hourStr = timeStr.substring(0, colonIndex);
String minuteStr = timeStr.substring(colonIndex + 1);

int startHour = hourStr.toInt();
int startMinute = minuteStr.toInt();
int durationMinutes = durationStr.toInt();

if (startHour < 0 || startHour > 23 || startMinute < 0 || startMinute > 59 || durationMinutes <= 0) {
  webSocket.sendTXT(num, "invalid values");
  return;
}

scheduledStartMinutes = startHour * 60 + startMinute;
scheduledDurationMinutes = durationMinutes;
scheduleEnabled = true;

String reply = "OK start set to ";
if (startHour < 10) reply += "0";
reply += String(startHour);
reply += ":";
if (startMinute < 10) reply += "0";
reply += String(startMinute);
reply += " for ";
reply += String(durationMinutes);
reply += " minutes";
```

```
      webSocket.sendTXT(num, reply);
    }
    else {
      webSocket.sendTXT(num, "unknown command");
    }
    break;
  }

  default:
    break;
 }
}

void setup() {
 Serial.begin(9600);
 I2CBus.begin(I2C_SDA, I2C_SCL);

 while (sht3x.begin() != 0) {
  Serial.println("Failed to initialize the chip, please confirm the chip connection");
  delay(1000);
 }

 Serial.print("chip serial number: ");
 Serial.println(sht3x.readSerialNumber());

 if (!sht3x.softReset()) {
  Serial.println("Failed to reset the chip");
 }
```

```
if (!sht3x.startPeriodicMode(sht3x.eMeasureFreq_Hz5)) {
  Serial.println("Failed to enter the periodic mode");
}
Serial.println("------------------Read data in cycle measurement mode-----------------------");

pinMode(34, INPUT);
pinMode(35, INPUT);
pinMode(32, INPUT);
pinMode(33, OUTPUT);
pinMode(25, OUTPUT);

SPI.begin(18, -1, 23, 5);

tft.initR(INITR_BLACKTAB);
tft.setRotation(1);
tft.fillScreen(ST77XX_BLACK);
tft.setTextWrap(false);
tft.setTextSize(1);
tft.setTextColor(ST77XX_WHITE, ST77XX_BLACK);

WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
}
Serial.println();
Serial.print("WiFi connected, IP: ");
Serial.println(WiFi.localIP());
```

```
  configTzTime("AST4ADT,M3.2.0,M11.1.0", "pool.ntp.org", "time.nist.gov");

  Serial.println("Waiting for network time...");
  struct tm timeinfo;
  int attempts = 0;
  while (!getLocalTime(&timeinfo) && attempts < 20) {
    Serial.print(".");
    delay(500);
    attempts++;
  }
  Serial.println();

  if (getLocalTime(&timeinfo)) {
    Serial.printf("Time synced: %02d:%02d:%02d\n",
                  timeinfo.tm_hour, timeinfo.tm_min, timeinfo.tm_sec);
  } else {
    Serial.println("Failed to get network time");
  }

  webSocket.begin();
  webSocket.onEvent(webSocketEvent);
}

void loop() {
  webSocket.loop();

  if (scheduleEnabled) {
    struct tm timeinfo;
    if (getLocalTime(&timeinfo)) {
```

```
    int currentMinutes = timeinfo.tm_hour * 60 + timeinfo.tm_min;
    int endMinutes = scheduledStartMinutes + scheduledDurationMinutes;

    bool lightsShouldBeOn;

    if (endMinutes < 1440) {
      lightsShouldBeOn =
        (currentMinutes >= scheduledStartMinutes && currentMinutes < endMinutes);
    } else {
      lightsShouldBeOn =
        (currentMinutes >= scheduledStartMinutes ||
         currentMinutes < (endMinutes % 1440));
    }

    lights_on = lightsShouldBeOn ? 1 : 0;

    Serial.printf("Current time: %02d:%02d, schedule start=%02d:%02d, duration=%d, lights_on=%d\n",
                  timeinfo.tm_hour,
                  timeinfo.tm_min,
                  scheduledStartMinutes / 60,
                  scheduledStartMinutes % 60,
                  scheduledDurationMinutes,
                  lights_on);
  }
}

int water_level = digitalRead(34);
int soil_1 = analogRead(35);
int soil_2 = analogRead(32);
```

```
int soil1Percent = moisturePercent(soil_1);
int soil2Percent = moisturePercent(soil_2);
float tempC = sht3x.getTemperatureC();
float humidity = sht3x.getHumidityRH();

unsigned long now = millis();

if (autoWaterEnabled) {
  if (wateringPulseActive) {
    if (now - wateringPulseStart >= wateringDurationMs) {
      water_on = 0;
      wateringPulseActive = false;
      lastWateringTime = now;
      Serial.println("Auto watering pulse finished");
    }
  } else {
    bool cooldownFinished = (now - lastWateringTime >= wateringCooldownMs);
    bool soilDry = (soil1Percent < moistureTriggerPercent || soil2Percent < moistureTriggerPercent);

    if (cooldownFinished && soilDry) {
      water_on = 1;
      wateringPulseActive = true;
      wateringPulseStart = now;
      Serial.println("Auto watering pulse started");
    }
  }
}

digitalWrite(33, lights_on);
```

```
digitalWrite(25, water_on);

Serial.print("Water level: ");
Serial.println(water_level);
Serial.print("Soil1 raw: ");
Serial.print(soil_1);
Serial.print("  Soil1 %: ");
Serial.println(soil1Percent);
Serial.print("Soil2 raw: ");
Serial.print(soil_2);
Serial.print("  Soil2 %: ");
Serial.println(soil2Percent);
Serial.print("Temp (C): ");
Serial.print(tempC);
Serial.print("  Humidity (%RH): ");
Serial.println(humidity);

struct tm timeinfo;

tft.setTextColor(ST77XX_WHITE, ST77XX_BLACK);

tft.setCursor(0, 0);
if (getLocalTime(&timeinfo)) {
  tft.print("Time: ");
  if (timeinfo.tm_hour < 10) tft.print("0");
  tft.print(timeinfo.tm_hour);
  tft.print(":");
  if (timeinfo.tm_min < 10) tft.print("0");
  tft.print(timeinfo.tm_min);
```

```
  tft.print("   ");
} else {
  tft.print("Time: unavailable   ");
}

tft.setCursor(0, 10);
tft.print("Water: ");
tft.print(water_level == 1 ? "YES" : "NO ");
tft.print("   ");

tft.setCursor(0, 20);
tft.print("Soil1: ");
tft.print(soil1Percent);
tft.print("%     ");

tft.setCursor(0, 30);
tft.print("Soil2: ");
tft.print(soil2Percent);
tft.print("%     ");

tft.setCursor(0, 40);
tft.print("TempC: ");
tft.print(tempC, 1);
tft.print("     ");

tft.setCursor(0, 50);
tft.print("Humid: ");
tft.print(humidity, 1);
tft.print("     ");
```

```
tft.setCursor(0, 60);
tft.print("Lights: ");
tft.print(lights_on ? "ON " : "OFF");
tft.print("   ");

tft.setCursor(0, 70);
if (wateringPulseActive) {
  tft.print("Pump: ON  ");
} else if (autoWaterEnabled) {
  tft.print("Pump: AUTO");
} else {
  tft.print("Pump: MAN ");
}

tft.setCursor(0, 80);
if (scheduleEnabled) {
  tft.print("Sched: ");
  int schedHour = scheduledStartMinutes / 60;
  int schedMin = scheduledStartMinutes % 60;
  if (schedHour < 10) tft.print("0");
  tft.print(schedHour);
  tft.print(":");
  if (schedMin < 10) tft.print("0");
  tft.print(schedMin);
  tft.print(" ");
  tft.print(scheduledDurationMinutes);
  tft.print("m   ");
} else {
```

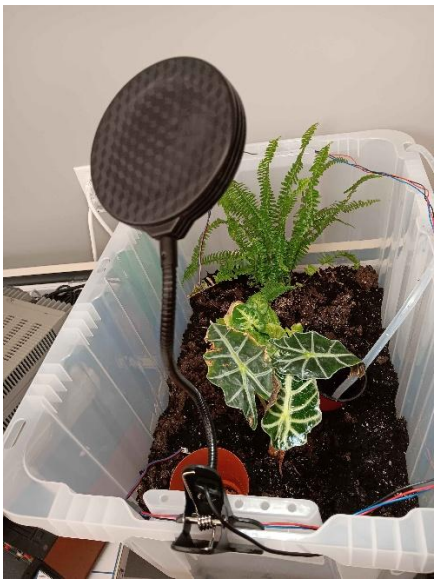
```
    tft.print("Sched: OFF        ");
  }

  String updateMsg = "";
  if (getLocalTime(&timeinfo)) {
    updateMsg += "time=";
    if (timeinfo.tm_hour < 10) updateMsg += "0";
    updateMsg += String(timeinfo.tm_hour);
    updateMsg += ":";
    if (timeinfo.tm_min < 10) updateMsg += "0";
    updateMsg += String(timeinfo.tm_min);
    updateMsg += ", ";
  }

  updateMsg += "water=" + String(water_level);
  updateMsg += ", pump_on=" + String(water_on);
  updateMsg += ", soil1_pct=" + String(soil1Percent);
  updateMsg += ", soil2_pct=" + String(soil2Percent);
  updateMsg += ", soil1_raw=" + String(soil_1);
  updateMsg += ", soil2_raw=" + String(soil_2);
  updateMsg += ", tempC=" + String(tempC, 1);
  updateMsg += ", humidity=" + String(humidity, 1);
  updateMsg += ", lights_on=" + String(lights_on);
  updateMsg += ", auto_water=" + String(autoWaterEnabled ? "1" : "0");

  webSocket.broadcastTXT(updateMsg);
  delay(1000);
}
```

APPENDIX D: USER MANUAL

The system mostly consists of a software terminal used to interact with system. Most of this user manual is going to consist of getting the software set up and making the connection between the terminal and the physical system.

1: The physical system:

- The light can be moved around to get the optimal distance from the plants.



- The soil moisture sensors can be moved around the enclosure

- The water pump can be positioned based on use case

2: Configuring the code:

```
// WiFi
const char* ssid = "";
const char* password = "";
```

The only change that will need to be made to the code is to put in a wifi SSID and password.

3: The Software and user interface

3.1 -WSCAT

wscat is a command-line WebSocket client. It establishes a connection to a server and provides an interface for transmitting and receiving messages over that connection. It is used in the project to be able to update the running conditions of the program without having to take it offline.

Installation:

In a powershell terminal run the following:

npm install -g wscat

Then, using the IP address of the plant automation system run:

wscat -c ws://$IP where $IP is the the address of the plant automation system.

Running:

Now from here there are a few basic commands:

lights on
lights off
water on
water off
start [time] [duration]

Examples:

lights on, lights off, water on, and water off all work as they sound.

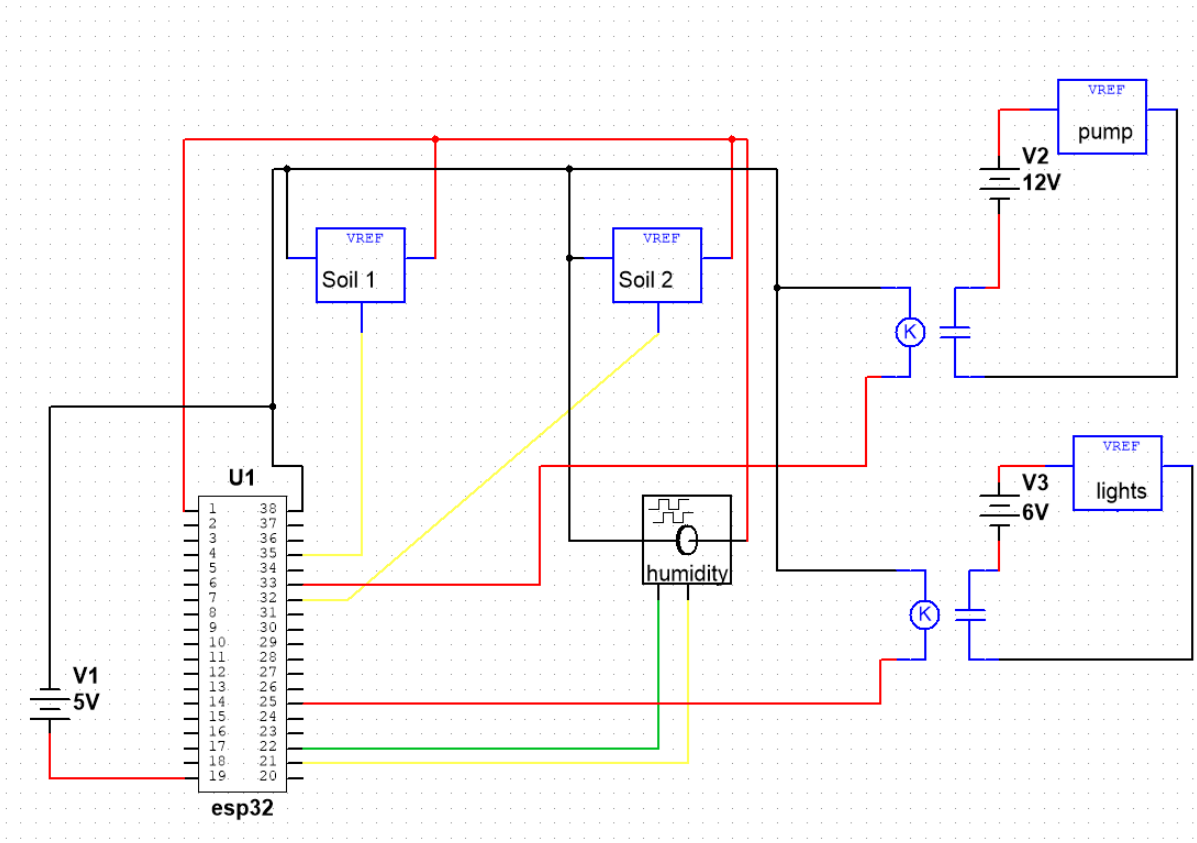
start 17:00 120 – this would turn the lights on at 17:00(5 PM) every day and would leave them on for 120 minutes. The system will set the time automatically based on the network time.

3.2 – TEE-OBJECT

Tee-Object is only used to be able to store the output of the terminal to a file while viewing it at the same time.

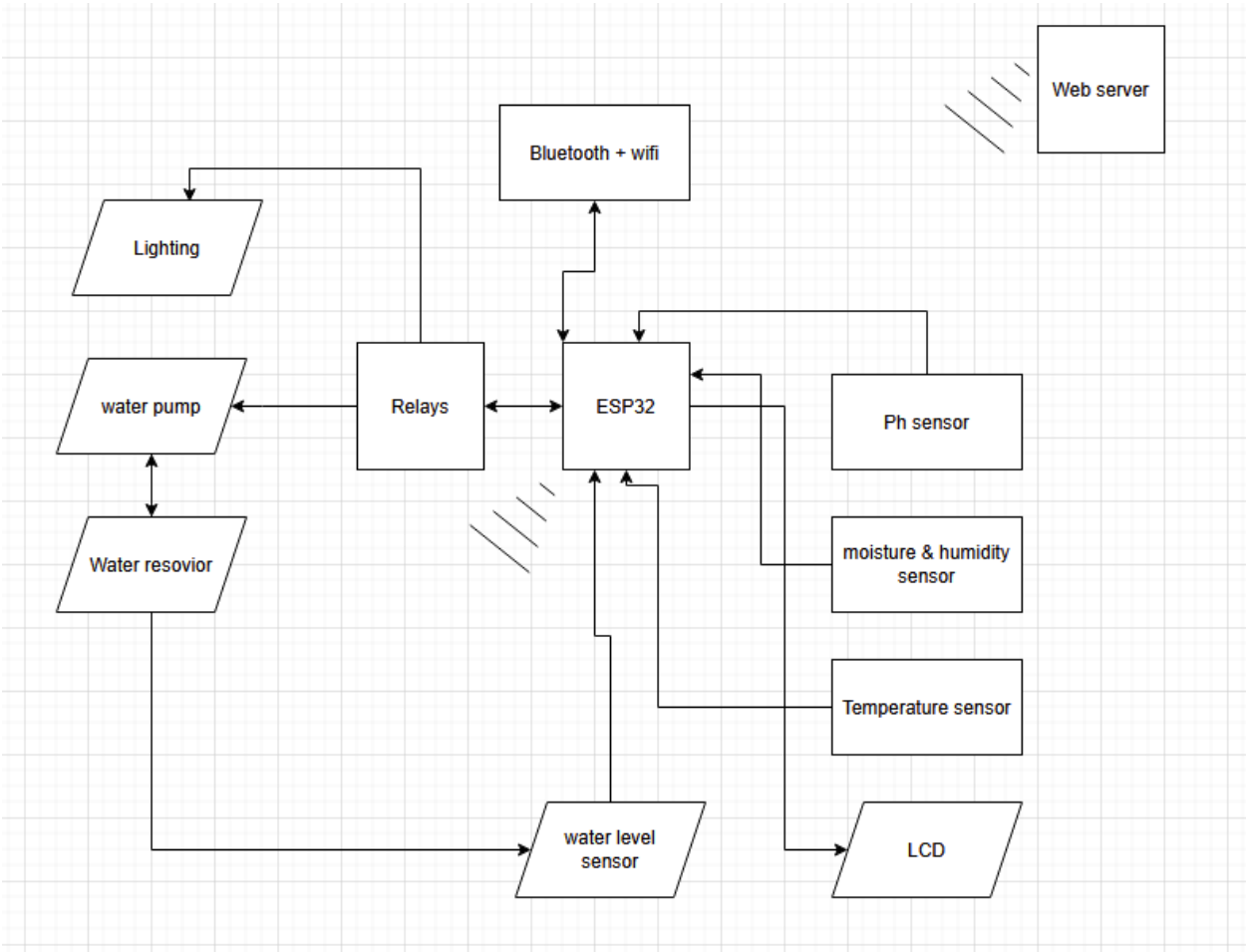
APPENDIX E: WIRING DIAGRAM AND BLOCK DIAGRAM

Wiring Diagram



The esp32 is run from the 5v usb port or can be energized through pin 19. This system isolates the water pump and lights by building three separate circuits that have their grounds separated. This is achieved through the two solid state relays.

The system can be modified to use almost all of the remaining pins as inputs or outputs. Some pins have to be in a certain state at boot, so some are recommended not to use. It would be safe to add more soil sensors or sensors for other functions, it would just be advised to make sure not to exceed the current limit for the esp32 pins. The soil and humidity sensors use around 5 mA each, which is why I'm powering them from the esp32. It could handle a few more, but it would make sense to power these sensors from the 5v supply directly, rather than through the 5v output pin on esp32.

In the wiring diagram the wire colors correspond to the wire colors for each device.

Block Diagram